HMRC: Heterogeneous Multi-Representative Centroids for Improved Vector Database Routing

Kishore Pulagam
Northwestern University
Evanston, IL, USA
kishore.pulagam@kellogg.northwestern.edu

Abstract

Vector databases partition vectors into segments and use centroids to route queries. When segments contain vectors from multiple topics/ domains, the mean centroid fails to represent any topic well, leading to routing errors. Our experiments on MS MARCO (100K passages) show that routing recall drops from 100% on coherent segments to 64.4% when segments mix 3 topics. This is often the case in enterprise content management systems and RAG systems, rendering the simple centroid based routing inefficient.

We address this with HMRC (Heterogeneous Multi-Representative Centroids), which uses k-means clustering to generate multiple centroids per segment. With 3 centroids per segment, routing recall improves from 64.4% to 81.2%—a 26% gain—with only 3x centroid storage overhead. Performance on coherent segments remains at 100%.

Keywords

vector databases, nearest neighbor search, routing, embeddings

1 Introduction

Vector databases are core to most retrieval-augmented generation (RAG) systems today. To scale, these systems partition vectors into segments and represent each segment by a centroid, typically the mean. Queries are first compared to centroids, and only the closest segments are searched. This routing step is critical: routing to the wrong segment means missing the target vector entirely.

There is a problem with this approach. When a segment contains vectors from multiple topics/domains, the mean centroid lands between topics and represents none of them well. A query about topic A may be closer to another segment's centroid than to the mean of a segment that actually contains topic A content.

Such heterogeneous segments are common in practice. Documents get batched by ingestion time, not by topic. Multi-tenant systems group vectors by customer. Incremental updates append to existing segments. All of these create segments with mixed content.

1.1 Contributions

We make three contributions:

(1) We quantify the heterogeneous segment problem on real data. Routing recall drops from 100% to 64.4% when segments mix just 3 topics.
(2) We propose HMRC, which uses k-means to generate multiple centroids per segment, capturing internal topic structure.
(3) We show HMRC improves routing recall by 26% on mixed segments without hurting coherent segments.

2 Related Work

IVF Indexing. FAISS [1] and similar systems cluster vectors and route queries to nearest clusters. The standard way to handle routing errors is increasing nprobe (searching more clusters), but this scales search cost linearly without fixing the representation problem.

SOAR. Google's ScaNN [2] assigns vectors to multiple clusters, improving recall at the cost of 2-3x vector storage. HMRC differs by operating at the segment level—we multiply centroid storage (negligible) rather than vector storage.

Graph Methods. HNSW [3] avoids centroids but requires more memory and is harder to distribute.

None of these approaches directly address segment-level centroid representation for heterogeneous segments.

3 Problem: Centroid Degradation

Consider a segment S containing vectors from two topic clusters with means $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$. The segment mean is:

$$\bar{\mathbf{c}} = \frac{|S_1|}{|S|}\boldsymbol{\mu}_1 + \frac{|S_2|}{|S|}\boldsymbol{\mu}_2 \tag{1}$$

This mean lies between the two clusters. A query near $\boldsymbol{\mu}_1$ may be closer to a different segment's centroid than to $\bar{\mathbf{c}}$, causing a routing miss.

3.1 Empirical Validation

We validated this on MS MARCO passages (100K passages, 384 dimensions, 200 segments of 500 vectors each). Table 1 shows routing recall under different segment compositions.

Table 1: Routing recall degrades with segment heterogeneity

Segment Type	Recall@10	Drop
Coherent (1 topic)	100.0%	—
Semi-3 (3 topics)	64.4%	-35.6%
Semi-5 (5 topics)	49.1%	-50.9%
Random	14.6%	-85.4%

Mixing 3 topics per segment—a realistic scenario for time-batched ingestion—drops recall by 35.6 percentage points. The problem is substantial even at moderate heterogeneity levels.

4 HMRC: Our Approach

HMRC represents each segment with multiple centroids instead of one. We use k-means clustering within each segment to discover these centroids.

4.1 Index Construction

For each segment S_j, we run k-means with k clusters (default $k = 3$). The resulting cluster centers become the segment's representatives. We store all representatives in a flat list with a mapping from each representative to its segment ID.

For small segments (fewer than $2k$ vectors), we fall back to the mean, as k-means would be unreliable.

Algorithm 1 HMRC Index Construction

Require: Segments $\{S_1, ..., S_m\}$, number of representatives k
for each segment S_j **do**
  if $|S_j| < 2k$ **then**
    $R_j \leftarrow \{\text{mean}(S_j)\}$
  else
    $R_j \leftarrow \text{KMeans}(S_j, k).\text{centers}$
  end if
  Store R_j with mapping to segment j
end for

4.2 Query Routing

Given a query, we compute distances to all representatives (across all segments), find the nearest ones, and return the corresponding segment IDs after deduplication.

Algorithm 2 HMRC Query Routing

Require: Query $\mathbf{q}$, all representatives R, segment mapping M
Compute distances from $\mathbf{q}$ to all $r \in R$
Sort representatives by distance
Collect segment IDs, skipping duplicates
return Top-k unique segment IDs

4.3 Overhead

Storage overhead is $k\times$ the baseline centroid storage. For 10K segments with 384-dimensional vectors:

- Baseline: 10K × 384 × 4 bytes = 15 MB
- HMRC-3: 30K × 384 × 4 bytes = 45 MB

This is negligible compared to storing the vectors themselves. Query-time overhead is also $k\times$ for the centroid comparisons, which is small relative to the segment search cost.

5 Experiments

5.1 Setup

We used MS MARCO passages embedded with all-MiniLM-L6-v2 (384 dimensions). We created segments under four conditions:

- **Coherent:** Segments formed by k-means clustering (one topic each)
- **Semi-k:** Segments mixing vectors from k random topic clusters
- **Random:** Vectors assigned to segments uniformly at random

We used 2,000 queries sampled from the corpus. The metric is Recall@10: the fraction of queries where the ground-truth segment appears in the top 10 routed segments.

5.2 Main Results

Table 2 shows performance on Semi-3 segments (the realistic mixed case).

Table 2: HMRC performance on Semi-3 segments

Method	Recall@10	Gain	Overhead
Baseline	64.4%	—	1x
HMRC-2	72.1%	+12%	2x
HMRC-3	81.2%	+26%	3x
HMRC-5	80.7%	+25%	5x
HMRC-7	80.8%	+26%	7x

HMRC-3 provides the best tradeoff: 26% improvement with 3x centroid storage. Going beyond $k = 3$ does not help because the segments only contain 3 topics.

5.3 Performance on Coherent Segments

A concern is whether HMRC might hurt performance when segments are already coherent. Table 3 shows this is not the case.

Table 3: HMRC across segment types

Method	Coherent	Semi-3	Random
Baseline	100.0%	64.4%	14.6%
HMRC-3	100.0%	81.2%	16.2%
HMRC-5	100.0%	80.7%	22.1%

On coherent segments, HMRC maintains 100% recall. The k-means centroids end up close together when the segment is already tight, so routing still works.

On random segments, improvement is minimal (14.6% to 16.2%). This is expected: random segments have no topic structure for k-means to discover. Such segments are rare in production.

5.4 Varying Heterogeneity

Figure **??** shows how HMRC benefit changes with heterogeneity level. The gap between baseline and HMRC-3 widens as segments become more heterogeneous.

5.5 Varying Segment Size

We tested segment sizes from 100 to 2,000 vectors. HMRC provides consistent gains across all sizes, with larger relative improvements on smaller segments. Smaller segments are more affected by topic mixing because they have less room to absorb diversity.

6 Discussion

When does HMRC help? HMRC helps when segments have internal structure that k-means can discover. This covers most production scenarios: time-batched ingestion, source-based grouping,

incremental updates. It does not help on uniformly random segments, but these are uncommon in practice.

Choosing k. We recommend $k = 3$ as a default. If segments are known to have higher heterogeneity, $k = 5$ may help. Even $k = 2$ provides meaningful improvement (12%) at minimal cost.

Integration. HMRC requires no changes to segment structure or search logic. The only modification is how centroids are computed and stored. Any IVF-style system can adopt it.

7 Conclusion

We identified a gap in vector database routing: heterogeneous segments are poorly represented by mean centroids, causing routing failures. On MS MARCO, mixing 3 topics per segment drops routing recall from 100% to 64.4%.

HMRC addresses this by using k-means to generate multiple centroids per segment. With $k = 3$, recall improves from 64.4% to 81.2%—a 26% gain—at 3x centroid storage cost. Coherent segments remain unaffected.

The approach is simple to implement and applies to any system using centroid-based routing with non-clustered segments.

References

[1] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[2] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *ICML*, pages 3887–3896, 2020.

[3] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using HNSW graphs. *IEEE TPAMI*, 42(4):824–836, 2018.

[4] Jianguo Wang et al. Milvus: A purpose-built vector data management system. In *SIGMOD*, pages 2614–2627, 2021.

[5] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 33:9459–9474, 2020.